

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #7 - Semana 8 – Libro C - Capitulo 4
Funciones y estructura de los programas

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 16/09/2024

Bibliografía:

B. W. Kernighan and D. M. Ritchie, *The C programming language*, 2nd ed. Pearson, 2002, pp. 62–82.

Abstract— *This paper provides an overview of fundamental C programming concepts including function declaration, definition, and usage. It highlights the modularity achieved through functions, the role of external and static variables, and the importance of proper initialization. Key topics include recursion, variable scope, and preprocessor directives. Example code illustrates the application of these concepts, demonstrating function use, variable management, and type conversion.*

I. Introducción a las funciones

Las funciones dividen tareas de cálculo grandes en partes más pequeñas, permitiendo la reutilización de código y facilitando cambios en el programa. En C, los programas suelen constar de muchas funciones pequeñas en lugar de pocas grandes, y estas funciones pueden residir en uno o más archivos de código fuente, los cuales se compilan por separado y se combinan con funciones de bibliotecas previamente compiladas.

II. Declaración y definición de funciones

La declaración de funciones en C ha sido modificada por el estándar ANSI para permitir la declaración de tipos de argumentos, lo que facilita la detección de errores por parte del compilador y realiza conversiones de tipo automáticamente. La sintaxis para declarar una función es:

```
return-type function-name(argument declarations);
```

Mientras que la definición de la función es:

```
return-type function-name(argument declarations) {  
    // cuerpo de la función  
}
```

III. Funciones Básicas

Una función en C se define con un tipo de retorno, un nombre y argumentos. Por ejemplo, en un programa que busca líneas que contienen un patrón específico, podemos tener funciones como `getline` para leer una línea y `strindex` para encontrar un patrón en una cadena. El programa principal usa estas funciones para cumplir la tarea deseada.

IV. Funciones que retornan tipos no enteros

En C, las funciones no están limitadas a retornar `int`. Por ejemplo, la función `atof` convierte una cadena en un valor `double`. La declaración de `atof` se vería así:

```
double atof(char s[]);
```

Y su definición:

```
double atof(char s[]) {
    // código para convertir cadena a double
}
```

Para usar `atof` correctamente, es importante declarar el tipo de retorno correctamente en la función que lo llama para evitar errores.

V. Variables externas

En el lenguaje C, las variables externas y funciones tienen un alcance global. Estas variables se definen fuera de cualquier función, haciéndolas potencialmente accesibles desde cualquier función dentro del mismo programa. A diferencia de las variables internas, que se definen dentro de funciones y tienen un alcance limitado a esas funciones, las variables externas pueden ser utilizadas en múltiples funciones. Las funciones en C también son externas por defecto.

Las variables externas permiten una forma de comunicación entre funciones sin necesidad de pasar datos explícitamente a través de argumentos. Aunque son útiles para compartir datos entre funciones, el uso excesivo puede llevar a una mala estructura del programa y a conexiones de datos excesivas entre funciones.

Un ejemplo práctico de variables externas es un programa de calculadora en notación polaca inversa, donde la pila de valores y la posición de la pila se definen como variables externas para que las funciones `push` y `pop` puedan acceder a ellas sin necesidad de pasarlas como parámetros.

VI. Reglas de Alcance

El alcance de una variable o función en C determina dónde puede ser utilizada dentro del código. Para variables automáticas declaradas dentro de una función, el alcance es local a esa función. Por el contrario, las variables externas tienen un alcance desde su declaración hasta el final del archivo de código fuente donde están definidas. Si una variable externa se usa en otro archivo, debe ser declarada como `extern` en ese archivo.

La declaración y definición de variables externas deben ser gestionadas cuidadosamente para evitar múltiples definiciones y problemas de inicialización. Una definición de variable externa crea el espacio de almacenamiento necesario, mientras que una declaración extern simplemente informa al compilador sobre la existencia de una variable definida en otro lugar.

VII. Archivos de cabecera

Para dividir un programa en varios archivos fuente, se utilizan archivos de cabecera (.h) para centralizar las declaraciones y definiciones compartidas. Esto asegura que las declaraciones y definiciones comunes estén disponibles en todos los archivos fuente que las necesiten, manteniendo una única copia actualizada. En un programa más grande, se podría necesitar un número mayor de archivos de cabecera para una mejor organización.

VIII. Variables estáticas

Las variables estáticas en C son aquellas cuyo alcance está limitado al archivo de código fuente en el que están definidas. Esto se especifica mediante la palabra clave static. Las variables estáticas permiten ocultar variables y funciones que no deben ser accesibles desde fuera del archivo, proporcionando una encapsulación adicional y evitando conflictos de nombres.

IX. Inicialización

La inicialización en C es el proceso de asignar un valor inicial a una variable al momento de su declaración. Existen reglas específicas para diferentes tipos de variables:

1. **Variables Externas y Estáticas:** Estas variables se inicializan automáticamente a cero si no se proporciona una inicialización explícita. La inicialización se realiza una sola vez, conceptualizando el momento antes de que comience la ejecución del programa. Los inicializadores deben ser expresiones constantes.
2. **Variables Automáticas y de Registro:** Estas variables no tienen un valor definido por defecto y se inicializan a valores indeterminados (basura). La inicialización puede ser cualquier expresión, no necesariamente constante, que puede incluir llamadas a funciones.
3. **Inicialización de Arrays:** Los arrays se pueden inicializar al proporcionar una lista de valores entre llaves. El tamaño del array se determina automáticamente si se omite. Si hay menos inicializadores que el tamaño especificado, los elementos restantes se inicializan a cero.
4. **Cadenas de Caracteres:** Las cadenas de caracteres se inicializan de manera especial con comillas dobles, que se traduce internamente en un array de caracteres con un terminador nulo ('\0').

X. Recursión

La recursión en C permite que una función se llame a sí misma, ya sea directa o indirectamente. Existen dos formas principales de usar recursión:

1. **Ejemplo Básico:** Para imprimir un número en formato decimal, una función recursiva puede descomponer el número en dígitos y luego imprimirlos en el orden correcto.
2. **Algoritmo de Quicksort:** Un ejemplo clásico de recursión es el algoritmo de ordenamiento Quicksort. Este algoritmo divide un array en dos subarrays basados en un elemento pivote y aplica recursivamente el mismo proceso a estos subarrays hasta que se alcanza una condición de parada.

XI. El preprocesador de C

El preprocesador en C realiza varias funciones antes de la compilación efectiva del código:

1. **Inclusión de Archivos:** Usando `#include`, se pueden incluir archivos de cabecera y otros archivos de código. Esto facilita la gestión de declaraciones y definiciones compartidas.
2. **Sustitución de Macros:** Con `#define`, se pueden crear macros para sustituir tokens con secuencias de texto. También se pueden definir macros con argumentos para realizar sustituciones más complejas.
3. **Inclusión Condicional:** El preprocesador permite incluir código condicionalmente usando directivas como `#if`, `#ifdef`, `#ifndef`, y `#else`. Esto es útil para incluir o excluir partes del código dependiendo de ciertas condiciones.

XII. Código de ejemplo

Archivo de cabecera: functions.h

```
#ifndef FUNCTIONS_H
#define FUNCTIONS_H

// Declaraciones de funciones

void printMessage(const char *message);

int factorial(int n);

double atof(const char s[]);

extern int globalCounter; // Variable externa

#endif // FUNCTIONS_H
```

Archivo de implementación: functions.c

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include "functions.h"

// Definición de variable externa
int globalCounter = 0;

// Definición de la función printMessage
void printMessage(const char *message) {
    printf("%s\n", message);
}

// Definición de la función factorial usando recursión
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

// Definición de la función atof (simplificada)
double atof(const char s[]) {
    return strtod(s, NULL); // Usa la función estándar para conversión de cadenas
}

```

Archivo principal: main.c

```

#include <stdio.h>
#include "functions.h"

// Variable estática
static int staticVar = 0;

// Función para inicializar un array
void initializeArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = i * 2; // Inicialización de array
    }
}

int main() {
    // Uso de variable externa
    globalCounter++;

    // Uso de funciones
    printMessage("Hello, World!");

    // Inicialización y uso de variables automáticas
    int num = 5;
    printf("Factorial of %d is %d\n", num, factorial(num));

    // Inicialización de un array
    int numbers[10];
    initializeArray(numbers, 10);
    for (int i = 0; i < 10; i++) {
        printf("numbers[%d] = %d\n", i, numbers[i]);
    }

    // Uso de atof
    const char *str = "123.456";
    double value = atof(str);
    printf("Converted value: %f\n", value);
}

```

```
    return 0;  
}
```

Explicación del código:

1. Archivos de Cabecera (functions.h):

- Se definen las declaraciones de funciones y la variable externa globalCounter.
- Se utilizan directivas de preprocesador (#ifndef, #define, #endif) para evitar la inclusión múltiple del archivo de cabecera.

2. Archivo de Implementación (functions.c):

- Se define la variable externa globalCounter, que se declara en functions.h.
- Se implementan las funciones printMessage, factorial (con recursión), y atof (simplificada usando la función estándar strtod).

3. Archivo Principal (main.c):

- Se utiliza la variable externa globalCounter.
- Se demuestra el uso de funciones, variables automáticas y estáticas.
- Se inicializa un array y se usa en el programa.
- Se convierte una cadena a un número flotante usando atof.